7.2 Combining Them

We can mix and match these two concepts. One element of a hash might be an array as in the previous example, or you can have an array of hashes. For example, a form might let you create any number of addresses by repeating the following form fragment

```
<input name="addresses[][line1]" type="text"/>
<input name="addresses[][line2]" type="text"/>
<input name="addresses[][city]" type="text"/>
```

This would result in `params[:addresses]` being an array of hashes with keys `line1`, `line2` and `city`. Rails decides to start accumulating values in a new hash whenever it encounters an input name that already exists in the current hash.

There's a restriction, however, while hashes can be nested arbitrarily, only one level of "arrayness" is allowed. Arrays can usually be replaced by hashes; for example, instead of having an array of model objects, one can have a hash of model objects keyed by their id, an array index or some other parameter.

Array parameters do not play well with the `check_box` helper. According to the HTML specification unchecked checkboxes submit no value. However it is often convenient for a checkbox to always submit a value. The `check_box` helper fakes this by creating an auxiliary hidden input with the same name. If the checkbox is unchecked only the hidden input is submitted and if it is checked then both are submitted but the value submitted by the checkbox takes precedence. When working with array parameters this duplicate submission will confuse Rails since duplicate input names are how it decides when to start a new array element. It is preferable to either use `check_box_tag` or to use hashes instead of arrays.

7.3 Using Form Helpers

The previous sections did not use the Rails form helpers at all. While you can craft the input names yourself and pass them directly to helpers such as `text_field_tag` Rails also provides higher level support. The two tools at your disposal here are the name parameter to `form_for` and `fields_for`and the `:index` option that helpers take.

You might want to render a form with a set of edit fields for each of a person's addresses. For example:

```
<%= form_for @person do |person_form| %>
  <%= person_form.text_field :name %>
  <% @person.addresses.each do |address| %>
    <%= person_form.fields_for address, index: address.id do
|address_form|%>
      <%= address_form.text_field :city %>
    <% end %>
  <% end %>
<% end %>
```

Assuming the person had two addresses, with ids 23 and 45 this would create output similar to this:

```
<form accept-charset="UTF-8" action="/people/1" class="edit_person"
id="edit_person_1" method="post">
  <input id="person_name" name="person[name]" type="text" />
  <input id="person_address_23_city" name="person[address][23][city]"
type="text" />
  <input id="person_address_45_city" name="person[address][45][city]"
type="text" />
</form>
```

This will result in a `params` hash that looks like

```
{'person' => {'name' => 'Bob', 'address' => {'23' => {'city' => 'Paris'},
'45' => {'city' => 'London'}}}}
```